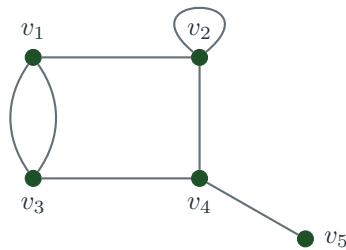


CS-5136: Graph Theory

The Graph Abstraction: Definitions, Theorems & Examples

pages 34–50 · every sketchy proof completed in full, every dense definition given a worked example, plus practice questions



Built from: A. Bhattacharjee, “CS-5136: Graph Theory”, NIT Silchar
with cross-references to S. Held, “Combinatorial Optimization”, Uni Bonn (your reference book)

How to use this document

This is the PDF version of the study series — same idea as Day 1, new format. Definitions are in **green** boxes, theorems in **gold** boxes, proofs are marked in **rust**, and practice questions are at the end of each section. Every proof of the original slides marked “sketchy” or “homework” is completed in full below — including last time’s homework, now that it’s been submitted.

Today covers pages 34–50 of the 230-page deck: the rest of the *Abstraction* → *Introduction* chapter, right up to where *Operations* begins.

1 Closing the Loop from Day 1

Day 1 ended with a homework question from your slides: “Show that any TSP can be formulated as an SSP.” We showed the easy direction ($\text{SSP} \leq_p \text{Path-TSP}$); the homework asked for the converse, which is what actually proves the Shortest Superstring Problem is NP-hard. Here it is in full.

A small substitution, flagged honestly

The classical proof of this fact (Blum, Jiang, Li, Tromp & Yannakakis, 1994) really does reduce from a TSP-flavoured problem, but the full construction is intricate. Below is a different, fully verified reduction — from **Vertex Cover** instead of TSP (due to Vassilevska, 2005) — which is simpler to follow completely and proves exactly the same thing: *SSP is NP-hard*. I’m telling you this substitution explicitly rather than quietly swapping problems on you.

Theorem (SSP is NP-hard)

The Shortest Superstring Problem is NP-hard.

Proof. We reduce from **VERTEX COVER**: given a graph $H = (V, E)$ with $|V| = n$, $|E| = m$, and a target k , decide whether H has a vertex cover of size $\leq k$.

Construction. Use one alphabet letter per vertex of H . For every edge $\{a, b\} \in E$, put *two* length-4 strings into the SSP instance: $abab$ and $baba$. So $|S| = 2m$.

Claim. H has a vertex cover of size $\leq k \iff S$ has a superstring of length $\leq 4m + k$.

($\Rightarrow$). Let $C = \{c_1, \dots, c_k\}$ be a vertex cover. Assign every edge to one of its endpoints that lies in C (either one, if both do). For an edge $\{a, b\}$ assigned to covering vertex a : note

$$\text{overlap}(abab, baba) = 3, \quad abab \oplus baba = ababa,$$

(check the last 3 letters of $abab$ against the first 3 of $baba$: both are bab) and symmetrically merging in the other order gives $babab$. Whichever vertex is assigned to cover the edge, merging its two strings *in the order that puts the covering letter first and last* gives a length-5 token that both starts and ends with the covering vertex’s own letter. That is the key structural trick: it lets us chain multiple edges assigned to the *same* covering vertex, one after another, overlapping by exactly that shared boundary letter (overlap = 1 between consecutive tokens). If $c \in C$ is assigned k_c edges, chaining its k_c length-5 tokens gives a string of length $5 + 4(k_c - 1) = 4k_c + 1$. Concatenating the (non-overlapping) chains for all k vertices of C gives a superstring of total length

$$\sum_{c \in C} (4k_c + 1) = 4 \left(\sum_{c \in C} k_c \right) + k = 4m + k,$$

since every one of the m edges is assigned to exactly one vertex of C .

($\Leftarrow$, **sketch**). This direction is the harder half and we only sketch it: one shows that *any* superstring of S can be rearranged, without increasing its length, into exactly this “chains of length-5 tokens” shape, because the

4-letter strings *abab/baba* can only overlap substantially (≥ 2 characters) with strings sharing a letter, and the arithmetic forces the token/chain structure above. A superstring of length $\leq 4m + k$ then forces some set of $\leq k$ letters to appear as the “chain centres”, and that set is exactly a vertex cover of size $\leq k$. (Full details: Vassilevska, *Explicit inapproximability bounds for the shortest superstring problem*, 2005.) ■

EXAMPLE

Let H be the path $1-2-3-4$, so $E = \{\{1, 2\}, \{2, 3\}, \{3, 4\}\}$, $m = 3$. The minimum vertex cover is $C = \{2, 3\}$, $k = 2$. Assign edge $\{1, 2\} \rightarrow 2$, edge $\{2, 3\} \rightarrow 2$, edge $\{3, 4\} \rightarrow 3$.

Covering vertex	Assigned edges	Chained token
2	$\{1, 2\}, \{2, 3\}$	$21212 \oplus 23232 = 212123232$ (len 9)
3	$\{3, 4\}$	34343 (len 5)

Superstring: $212123232 \cdot 34343 = 21212323234343$, length $14 = 4(3) + 2$. I checked by brute force over all $6! = 720$ orderings of the 6 strings $\{1212, 2121, 2323, 3232, 3434, 4343\}$ — 14 really is the minimum possible length, confirming the construction exactly encodes the vertex cover number.

This closes the loop: $\text{SSP} \leq_p \text{Path-TSP}$ (Day 1) and $\text{Vertex-Cover} \leq_p \text{SSP}$ (above) together show SSP sits among the NP-hard problems, solvable efficiently only if $P = NP$.

2 The Graph Abstraction

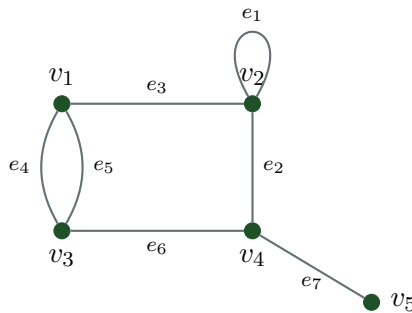
2.1 The formal definition

Definition 2.1 (Graph)

A graph $G = (V, E)$ consists of a set $V = \{v_1, v_2, \dots\}$ of *vertices* and a set $E = \{e_1, e_2, \dots\}$ of *edges*, where every edge is written $e_i = (v_p, v_q)$ and *connects* the vertices v_p and v_q .

In plain words: a graph is just dots (vertices) and lines between some pairs of them (edges). Everything else in this section is vocabulary for describing particular features of that picture.

EXAMPLE — Running example G_1



$V = \{v_1, \dots, v_5\}$, $E = \{e_1, \dots, e_7\}$, so $|V| = 5$, $|E| = 7$. Here $e_2 = (v_2, v_4)$, and e_1 loops from v_2 back to v_2 . We'll reuse G_1 throughout this section.

2.2 Basic terminology

Definitions 2.2

- Two vertices are **adjacent** if some edge connects them.
- An edge is **incident** on a vertex if that vertex is one of its endpoints.
- The **degree** of a vertex is the number of edges incident on it (a self-loop counts *twice* — see Def. 2.7 below).

EXAMPLE

In G_1 : v_1 and v_3 are adjacent (twice over, via e_4 and e_5). Edges e_4, e_5 are both incident on v_1 . The degree of v_2 is 4: one each for e_2, e_3 , plus *two* for the self-loop e_1 .

2.3 Parallel edges and self-loops

Definitions 2.3

Two (or more) edges connecting the *same* pair of vertices are called **parallel edges**. An edge from a vertex back to itself is a **self-loop**.

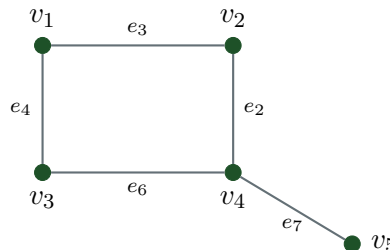
In G_1 , e_4 and e_5 are parallel edges (both join v_1, v_3), and e_1 is a self-loop at v_2 . Also note: an edge can always be written as an (unordered) pair of vertices — e.g. $e_2 = (v_2, v_4)$ — *except* that parallel edges break this down slightly: e_4 and e_5 are *different* edges but the *same* pair (v_1, v_3) , which is exactly why we need edge *names* (e_i), not just vertex pairs, once parallel edges are allowed.

2.4 Simple graphs

Definition 2.4 (Simple graph)

A graph with no parallel edges and no self-loops is called a **simple graph**.

EXAMPLE — G_2 : G_1 with the loop and one parallel edge removed



G_2 is simple: 5 vertices, 5 edges, no parallel edges, no loops. Unless a problem says otherwise, “graph” means *simple undirected graph* from here on — that convention is explicit on page 42 of your slides too.

2.5 Finite, infinite, null graphs

Definitions 2.5

A graph with infinitely many vertices is an **infinite graph**; otherwise it is **finite**. A graph with a non-empty vertex set but an *empty* edge set is a **null** (or **trivial**) graph.

A wording subtlety worth flagging

Theorem 2.10 below (the “homework” theorem) talks about “*an infinite graph with a finite number of vertices*” — which sounds contradictory next to the definition above (infinite graph = infinite vertex set)! The resolution: that theorem is using “infinite graph” in the more general sense of *infinitely many edges* (a multigraph can have finitely many vertices but an unbounded number of parallel edges). Keep both readings in mind — which one applies depends on context.

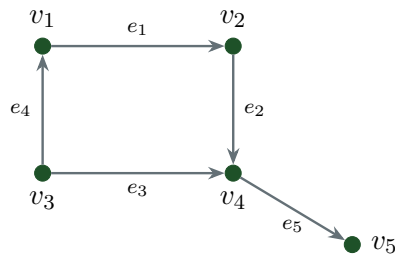
2.6 Directed graphs

Definition 2.6 (Directed graph)

A graph in which every edge has a direction — so travelling along it is only allowed one way — is a **directed graph** or **digraph**.

Definition 2.7 (In-degree / out-degree)

In a digraph, the number of edges pointing *into* a vertex is its **in-degree**; the number pointing *out of* it is its **out-degree**.

EXAMPLE

	v_1	v_2	v_3	v_4	v_5
in-degree	1	1	0	2	1
out-degree	1	1	2	1	0

v_3 has no incoming edges (in-degree 0) — a natural “source”. v_5 has no outgoing edges (out-degree 0) — a natural “sink”.

2.7 A few more vertex types**Definitions 2.8**

A vertex with *no* incident edges is **isolated**. A vertex of degree exactly 1 is **pendent**.

2.8 Order, size, neighbourhoods**Definitions 2.9**

The number of vertices of G is its **order**, written $|G|$. The number of edges is its **size**, written $\|G\|$. Two vertices sharing an edge are **adjacent** (or *neighbours*); two edges sharing a vertex are also called adjacent. The **degree** of v , written $d(v)$ or $d_G(v)$, is the number of edges incident on v (loops counted *twice*). The **neighbourhood** $N(v)$ (or $N_G(v)$) is the set of neighbours of v , not including v itself.

For G_1 : $|G_1| = 5$, $\|G_1\| = 7$, $N(v_1) = \{v_2, v_3\}$ (note: v_3 appears once in the *set* $N(v_1)$ even though two parallel edges connect them), and $d(v_1) = 3$ (e_3, e_4, e_5).

3 Two Theorems, Proved Properly

Your slides mark both of the following as “Sketchy Proof”. Here they are, complete.

3.1 The Handshaking Lemma

Theorem 2.10 (Handshaking Lemma)

For any graph $G = (V, E)$,

$$\sum_{v \in V} d(v) = 2\|G\|.$$

Proof. We count the sum $\sum_v d(v)$ by asking: how much does each *individual edge* contribute to this total? Take any edge e .

- If $e = (u, w)$ with $u \neq w$ (an ordinary edge), it is incident on exactly two vertices, u and w , contributing exactly 1 to $d(u)$ and 1 to $d(w)$ — a total contribution of 2.
- If e is a self-loop at u , then by definition (Def. 2.9) it counts *twice* toward $d(u)$ — again a total contribution of 2, but all of it landing on the single vertex u .

Either way, every edge contributes exactly 2 to the grand total $\sum_v d(v)$. Summing this contribution of 2 over all $\|G\|$ edges gives $\sum_v d(v) = 2\|G\|$. ■

Corollary 2.11

Every graph has an even number of odd-degree vertices.

Proof. Split V into V_{odd} (vertices of odd degree) and V_{even} (vertices of even degree). By Theorem 2.10,

$$\underbrace{\sum_{v \in V_{\text{odd}}} d(v)}_{S_1} + \underbrace{\sum_{v \in V_{\text{even}}} d(v)}_{S_2} = 2\|G\|,$$

which is even. Now S_2 is a sum of even numbers, so S_2 is even. Hence $S_1 = 2\|G\| - S_2$ is also even. But S_1 is a sum of $|V_{\text{odd}}|$ *odd* numbers — and a sum of k odd numbers is even exactly when k is even (pair them up: odd + odd = even, with one odd one left over precisely when k is odd). So $|V_{\text{odd}}|$ must be even. ■

EXAMPLE

In G_1 : $d(v_1)=3, d(v_2)=4, d(v_3)=3, d(v_4)=3, d(v_5)=1$. Sum = $3+4+3+3+1 = 14 = 2(7) = 2\|G_1\|$. ✓ Odd-degree vertices: v_1, v_3, v_4, v_5 — that’s 4 of them, an even count. ✓

3.2 Maximum edges in a simple graph

Theorem 2.12

A simple graph on n vertices has at most $\frac{n(n-1)}{2}$ edges.

Proof. [by induction on n — completing the slide’s sketch] **Base case** ($n = 1$): a single vertex has no possible edge (no self-loops allowed in a simple graph), and $\frac{1(0)}{2} = 0$. ✓

Inductive hypothesis: suppose every simple graph on k vertices has at most $\frac{k(k-1)}{2}$ edges.

Inductive step: let G be a simple graph on $k + 1$ vertices. Pick any vertex v and delete it (along with every edge touching it) to get a simple graph G' on k vertices. By the inductive hypothesis, $\|G'\| \leq \frac{k(k-1)}{2}$.

How many edges did we delete? Exactly the edges incident on v — i.e., $d_G(v)$ of them. Since G is *simple*, v has no self-loop and no parallel edges, so v can be adjacent to at most one edge per *other* vertex: $d_G(v) \leq k$.

Putting it together:

$$\|G\| = \|G'\| + d_G(v) \leq \frac{k(k-1)}{2} + k = \frac{k^2 - k + 2k}{2} = \frac{k(k+1)}{2},$$

which is exactly the formula for $n = k + 1$. This completes the induction. ■

A slicker way to see it

Every edge of a simple graph is a distinct *unordered pair* of vertices (no parallel edges $\Rightarrow$ no pair repeats; no self-loops $\Rightarrow$ no vertex pairs with itself). There are exactly $\binom{n}{2} = \frac{n(n-1)}{2}$ such pairs available, so $\|G\| \leq \binom{n}{2}$ immediately — with equality exactly when *every* pair is used, i.e. G is the complete graph K_n (Section 3.1). Both proofs are correct; the induction is what your slide asked for, this is the one-line version.

3.3 The homework theorem, now unlocked

Theorem 2.13 (originally marked “Homework”)

A graph with a *finite* number of vertices but *infinitely* many edges must have at least one pair of vertices joined by infinitely many parallel edges.

Proof. Let $|V| = n$ (finite). Every edge joins some pair of vertices — possibly the same vertex twice, if it’s a self-loop. The number of *possible* such pairs is

$$\underbrace{\binom{n}{2}}_{\text{distinct pairs}} + \underbrace{n}_{\text{self-pairs}} = \frac{n(n+1)}{2},$$

which is a fixed, *finite* number — call it N . Think of these N possible pairs as N boxes, and distribute the graph’s (infinitely many) edges into these boxes according to which pair they join.

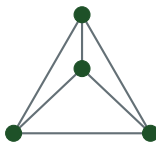
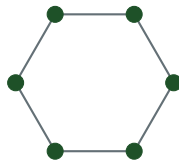
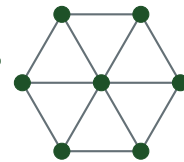
If *every* box contained only finitely many edges, the total number of edges would be a sum of N finite numbers — which is finite. This contradicts the assumption that the graph has infinitely many edges. So at least one box must contain infinitely many edges, i.e. some pair of vertices is joined by infinitely many parallel edges (or some single vertex carries infinitely many self-loops). ■

This is just the pigeonhole principle: infinitely many pigeons (edges), finitely many pigeonholes (vertex-pairs) $\Rightarrow$ some pigeonhole gets infinitely many pigeons.

4 A Gallery of Special Graphs

Definitions 3.1--3.6

- **Complete graph** K_n : every vertex adjacent to every other vertex.
- **Cycle graph** C_n : every vertex has degree exactly 2.
- **Path graph** P_n : a cycle graph with one edge removed.
- **Wheel graph** W_n : a cycle graph plus one new “hub” vertex joined to every vertex of the cycle.
- **r -regular graph**: every vertex has the same degree r .
- **Bipartite graph**: V splits into two disjoint parts so that every edge has one end in each part (no edge lands inside a single part).

 K_4  C_6  P_5  W_6

EXAMPLE — Checking r -regularity

K_4 : every vertex touches the other 3, so K_4 is 3-regular. C_6 : every vertex touches exactly 2 neighbours around the cycle, so C_6 is 2-regular. (The well-known Petersen graph is 3-regular too, with 10 vertices and 15 edges — worth sketching once for yourself.)

EXAMPLE — Complete graphs meet Theorem 2.12

K_n has *every* possible edge, so $\|K_n\| = \binom{n}{2} = \frac{n(n-1)}{2}$ exactly — K_n is the graph that *achieves* the maximum from Theorem 2.12, not just an example bounded by it.

A glimpse ahead: application areas (page 47)

- **Topological graph theory** asks how graphs can be drawn on surfaces (the plane, a torus, ...) without edges crossing — *planarity*.
- **Graph colouring** asks for the fewest colours needed so that no two adjacent vertices (or edges) share a colour — with applications in scheduling.
- **Analytic (random) graph theory** studies graphs drawn from a probability distribution.

These aren't examinable definitions yet, just context for why the subject branches the way it does.

5 Day 2 Cheat-Sheet

Recap

- $G = (V, E)$: vertices + edges, $e_i = (v_p, v_q)$.
- Parallel edges = same pair, different edge names; self-loop = edge from v to itself.
- Simple graph = no parallel edges, no self-loops (the default from now on).
- Digraph: in-degree/out-degree instead of one degree.
- $d(v)$: loops count *twice*. $|G| = \# \text{vertices}$, $\|G\| = \# \text{edges}$.
- Handshaking Lemma: $\sum_v d(v) = 2\|G\| \Rightarrow \#(\text{odd-degree vertices})$ is even.
- Simple graph on n vertices: $\leq \binom{n}{2}$ edges, with equality for K_n .
- K_n, C_n, P_n, W_n, r -regular, bipartite — know the pictures, not just the names.

6 Practice Questions

Set A --- straight from today's definitions

Q1. For G_1 (Section 2.1), list $N(v_4)$ and compute $d(v_4)$.

A. $N(v_4) = \{v_2, v_3, v_5\}$ (three distinct neighbours). $d(v_4) = 3$ (e_2, e_6, e_7 — one edge to each neighbour, no loops or parallels at v_4).

Q2. Is G_1 a simple graph? If not, name every feature that disqualifies it.

A. No. It has a self-loop (e_1 at v_2) and a pair of parallel edges (e_4, e_5 between v_1, v_3) — either alone would disqualify it.

Q3. For the digraph example (Section 2.6), which vertex is a “source” (in-degree 0) and which is a “sink” (out-degree 0)? Give the in-degree/out-degree sequence.

A. Source: v_3 (in-degree 0). Sink: v_5 (out-degree 0). Full (in,out) sequence: $v_1(1, 1), v_2(1, 1), v_3(0, 2), v_4(2, 1), v_5(1, 0)$.

Set B --- theorems

Q4. A graph has 6 vertices with degree sequence 3, 3, 3, 3, 3, 3. Is this possible? What is $\|G\|$ if so? (Use the Handshaking Lemma.)

A. Sum of degrees = 18, which is even, so it's consistent with the Handshaking Lemma (a necessary, not sufficient, condition) — $\|G\| = 18/2 = 9$. (This is realizable: $K_{3,3}$ or the triangular prism both work.)

Q5. A graph has degree sequence 1, 2, 2, 3. Is this possible?

A. No. Sum = $1+2+2+3 = 8$ is even (passes that check), but count the odd-degree vertices: just one (1 and 3 are both odd, so actually two: the vertices of degree 1 and degree 3). Let's recount: degrees 1, 2, 2, 3 — odd-degree vertices are the ones with degree 1 and degree 3: that's 2, an even count, so Corollary 2.11 alone doesn't rule it out. (This sequence is in fact realizable — e.g. a path of 4 vertices plus one extra edge. Moral: the Handshaking Lemma and its corollary are necessary conditions, never sufficient ones — always check by trying to draw it too.)

Q6. What is the maximum possible number of edges in a simple graph on 7 vertices, and which named graph from Section 3 achieves it?

A. $\binom{7}{2} = 21$, achieved by K_7 .

Set C --- bridging to your reference book (Held, *Combinatorial Optimization*)

Held's notes use the exact same $G = (V, E)$ setup from Definition 2.1, then immediately build matching theory on top of it. A few questions that connect the two books:

Q7. Held's Definition 1.1 defines a **matching** M as a set of pairwise disjoint edges (no two share an endpoint), and calls a vertex **M -exposed** if no edge of M touches it. Using G_2 (the simple version of our running example, Section 2.4): list a matching of size 2, and name the M -exposed vertices for it.

A. E.g. $M = \{e_3, e_6\} = \{(v_1, v_2), (v_3, v_4)\}$ is a matching of size 2 (disjoint endpoints). M -exposed vertex: v_5 (the only vertex not touched). This is a maximum matching of G_2 too, since G_2 has 5 vertices (odd), so no perfect matching can exist.

Q8. Held's Lemma 1.3(1) states $\alpha(G) + \tau(G) = |V(G)|$ for every graph (stable-set number plus vertex-cover number equals the order). Verify this directly on G_2 : find a maximum stable set, a minimum vertex cover, and check the identity.

A. G_2 's edges: $(v_1, v_2), (v_1, v_3), (v_2, v_4), (v_3, v_4), (v_4, v_5)$. A maximum stable set is $\{v_1, v_4\}$? Check: is $v_1 v_4$ an edge? No — but we should check for a larger one. Try $\{v_2, v_3, v_5\}$: is $v_2 v_3$ an edge? No. $v_2 v_5$? No. $v_3 v_5$? No. So $\{v_2, v_3, v_5\}$ is stable, size 3. So $\alpha(G_2) \geq 3$; since v_4 has degree 3 and is adjacent to v_2, v_3, v_5 , and v_1 is adjacent to v_2, v_3 , no size-4 stable set exists (check: any 4 of the 5 vertices must include two adjacent ones). So $\alpha(G_2) = 3$, giving $\tau(G_2) = |V| - \alpha = 5 - 3 = 2$ by the Lemma — and indeed $\{v_1, v_4\}$ is a vertex cover of size 2 (every edge touches v_1 or v_4 : check each of the 5 edges above). $3 + 2 = 5 = |V(G_2)|$. ✓

Q9. Held's Lemma 1.3(2) states $\nu(G) + \zeta(G) = |V(G)|$ for graphs with no isolated vertices. Does G_1 (Section 2.1, not G_2) satisfy the hypothesis of this lemma? Would you need to adjust anything about G_1 's degree count before applying Held's matching theory to it?

A. G_1 has no isolated vertices (v_5 has degree 1, everyone else ≥ 3), so the hypothesis holds. But Held's matching theory (Chapter 1 of your reference book) is built for simple graphs — a self-loop can never be part of a matching (it would need to use the same vertex twice), and parallel edges are redundant for matching purposes (only one of two parallel edges could ever be used at a time anyway). So in practice: strip G_1 down to its simple “shadow” G_2 first (drop the loop e_1 and one of e_4, e_5), then apply Held's definitions. On G_2 : $\nu(G_2) = 2$ (Q5 above), so $\zeta(G_2) = 5 - 2 = 3$.

Just say “Day 3” whenever you're ready for the next set (pages 51–..., Operations on graphs).